

question 2:

```
#include "ECTime.h"
#include <iostream>
using namespace std;

//your code goes here

ECTime::ECTime(int h, int m, int s) {
    hour = h;
    min = m;
    sec = s;
}

ECTime::~ECTime() {}

void ECTime::GetTime(int &h, int &m, int &s) const {
    h = hour;
    m = min;
    s = sec;
}

ECTime ECTime::operator+(const ECTime &tmToAdd) {
    int totalSec = sec + tmToAdd.sec;
    int totalMin = min + tmToAdd.min;
    int totalHour = hour + tmToAdd.hour;

    if (totalSec >= 60) {
        totalSec -= 60;
        totalMin += 1;
    }

    if (totalMin >= 60) {
        totalMin -= 60;
        totalHour += 1;
    }

    return ECTime(totalHour, totalMin, totalSec);
}

int main()
{
    ECTime t1(0, 1, 30), t2(1, 2, 31);
    ECTime t3 = t1 + t2;
    int s, m, h;
    t3.GetTime(h, m, s);
    cout << "h: " << h << ", m: " << m << ", s: " << s << endl;
}
```

```
karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab6
$ clang++ ECTime.cpp

karin@Karin's Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab6
$ ./a.exe
h: 1, m: 4, s: 1
```

question 4:

```
#include <cstdlib>
#include <cmath>

// Created by Y. Wu, Feb 27, 2023
// Base class: Shape for different kinds of geometry
// One interface function: CalcArea(), no parameter, return int (area of the shape). How
// should you define it?
template <typename T>
class Shape {
public:
    virtual ~Shape() = default;
    virtual T CalcArea() const = 0;
};

// Rectangle class: define two constructors
// (i) take two parameters, width and height
// (ii) take four parameters: x1,y1,x2,y2 where (x1,y1) is the upper left corner and (x2,y2)
// is the lower right corner
template <typename T>
class Rectangle : public Shape<T> {
public:
    Rectangle(T width, T height)
        : w(width >= 0 ? width : -width), h(height >= 0 ? height : -height) {}

    Rectangle(T x1, T y1, T x2, T y2)
        : w(std::abs(x2 - x1)), h(std::abs(y2 - y1)) {}

    T CalcArea() const override { return w * h; }

    T Width() const { return w; }
    T Height() const { return h; }

private:
    T w{0};
    T h{0};
};

// Class Square: a special rectangle
// One constructor: side length
template <typename T>
class Square : public Rectangle<T> {
public:
    explicit Square(T side) : Rectangle<T>(side, side) {}
};
```

```

#include "ECShapes.cpp"
#include <iostream>
using namespace std;

template <typename T>
void Area(Shape<T> &s) {
    cout << "Area: " << s.CalcArea() << endl;
}

void Test1() {
    Rectangle<int> r1(10,5);
    Area(r1);
    Square<int> s1(10);
    Area(s1);
}

// now test with custom object
// MyNum class has a custom constructor that takes a single integer
// Essentially MyNum is just like an integer and so it is not very useful on its own
// but we just want to test out template class
class MyNum {
public:
    MyNum(int x) : xv(x) {}
    // WHAT ELSE NEEDED?
    bool operator>=(const MyNum &other) const { return xv >= other.xv; }
    MyNum operator*(const MyNum &other) const { return MyNum(xv * other.xv); }
    MyNum operator-(const MyNum &other) const { return MyNum(xv - other.xv); }
    MyNum operator-() const { return MyNum(-xv); }
    friend MyNum abs(const MyNum &num) { return (num.xv < 0) ? MyNum(-num.xv) : num; }
    friend ostream &operator<<(ostream &os, const MyNum &num) {
        os << num.xv;
        return os;
    }
private:
    int xv;
};

void Test2() {
    cout << "Test2\n";
    MyNum n1(10), n2(5);
    Rectangle<MyNum> r(n1, n2);
    Area(r);
}

int main() {
    Test1();
    Test2();
}

```

```

karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab6
$ clang++ ECShapesTest.cpp

karin@Karin's_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab6
$ ./a.exe
Area: 50
Area: 100
Test2
Area: 50

```